TRON 2 SDK 开发指南

版本	修订日期	修订人	修改内容
V0.1	20250826	Eleven	初稿
v0.2	20250909	Damon	大纲调整
V0.3	20251011	Charlie	更新采集软件包，在「5.5.5 HDF5 数据存储格式统计表格」部分增加关节目标位置和夹爪目标位置
v0.4	20251119	Damon	增加3.5.3ServoJ 控制指令、3.5.4获取双臂末端位姿、3.5.5获取机器人关节状态
v0.5	20260113	Damon	固件版本更新至20260113的最新版本

1. TRON2 SDK 概述

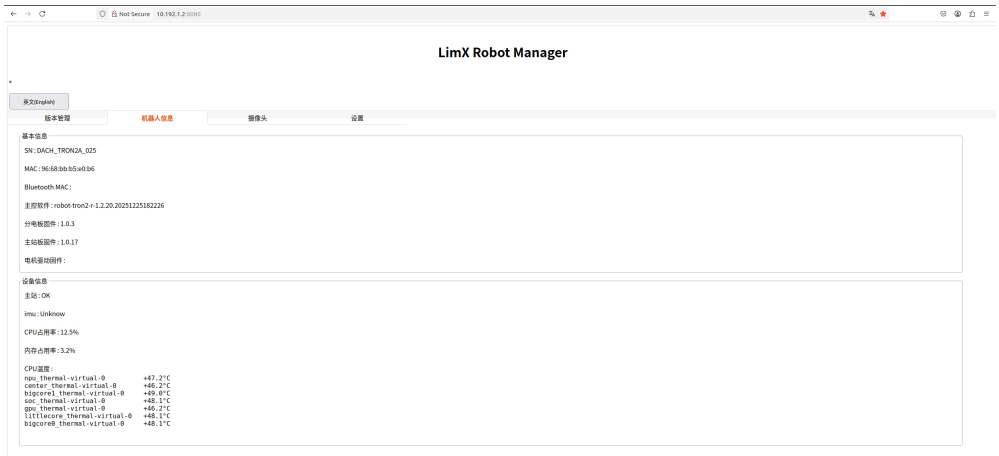
- **底层运动控制开发接口**：在开发者模式下，用户可以使用此接口开发自己的运动控制算法，完成算法的仿真和实机无缝部署。
- **上层应用开发接口**：在开发者模式下，用户可以在机器人预先安装的底层运动控制算法基础上，利用上层应用开发接口开发自己的软件业务功能，例如双臂操作和机器人管理等。

1.1 查看/设置机器人型号

在编译和运行控制算法及仿真器程序时，选择正确的机器人型号至关重要。您可以通过查看机器人型号并将其设置到环境变量 `ROBOT_TYPE` 中，确保在不同任务中准确识别并应用相应的机器人模型。以下是查看和配置机器人型号的步骤。

1.1.1 查看机器人型号

1. 请选择并连接您机器人的 Wi-Fi 热点，名称为 `XXXX_TRON2A_XXX_5G` 或 `XXXX_TRON2A_XXX_2.4G`，密码为：`12345678`
2. 在浏览器中输入 `http://10.192.1.2:8080` 可以进入“机器人信息页”，并查看机器人信息。如下图所示，页面中显示的SN (序列号) 为DACH_TRON2A_025，其中DACH_TRON2A即为机器人型号。



1.1.2 设置机器人型号

打开 Bash 终端，输入以下 Shell 命令来设置机器人型号。这样在编译和运行运动控制算法以及仿真器程序时，就能获取到正确的机器人型号信息。

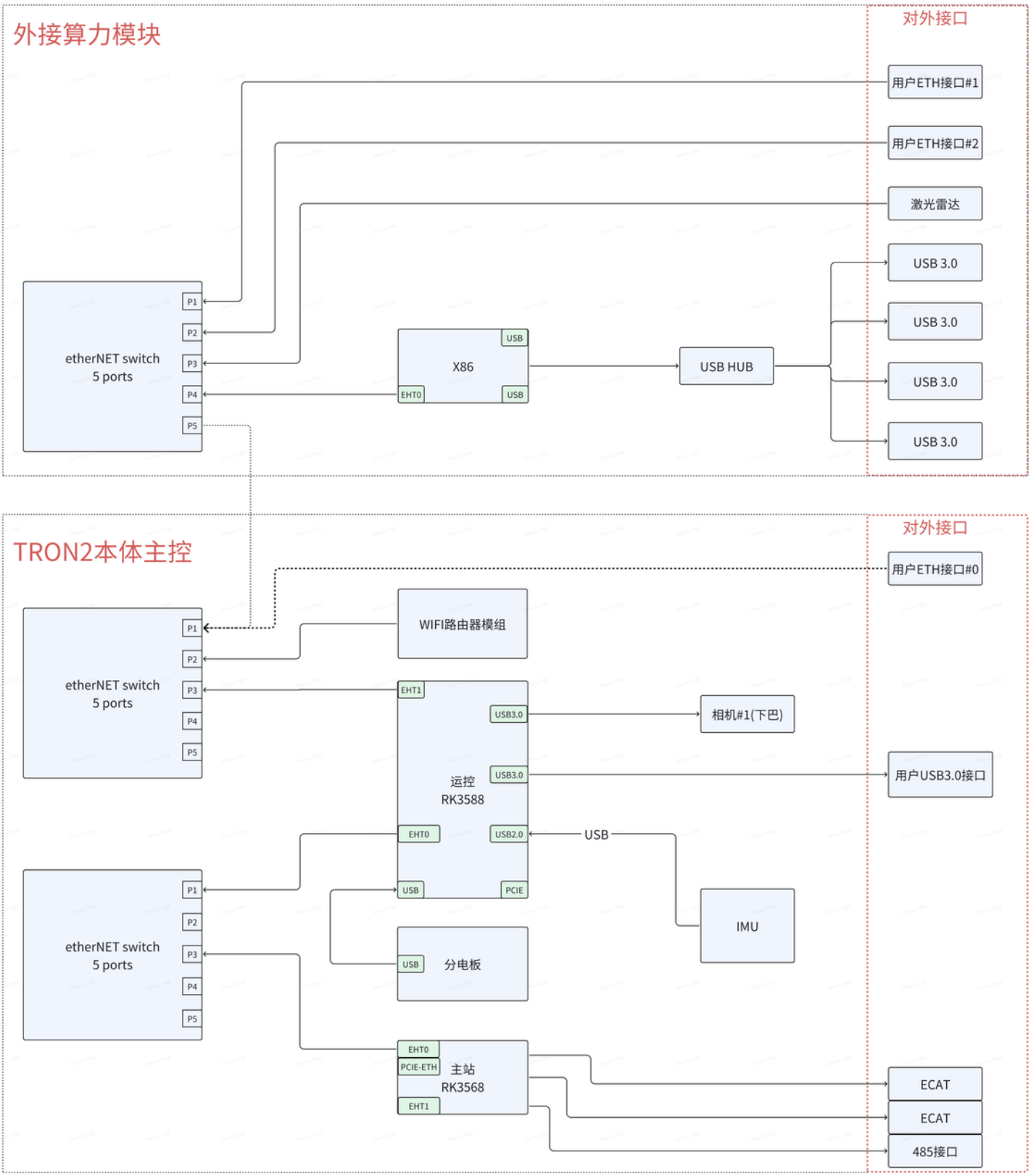
代码块

```
echo 'export ROBOT_TYPE=DACH_TRON2A' >> ~/.bashrc && source ~/.bashrc
```

1.2 通讯架构图

下图呈现了开发者的电脑与机器人本体的系统组成及交互关系。开发电脑部分涵盖运控算法节点和软件业务逻辑实现模块，通过 `上层应用开发接口` 和 `底层运动控制开发接口` 的数据通讯控制机器人

本体的运动。机器人本体由数据交换机、主控电脑及各类硬件组件构成，主控电脑负责协调各组件运行。



1.3 算力模块电脑

算力模块电脑主要用于开发机器人相关算法及应用程序。您可以通过 WiFi或有线网络连接到机器人本体系统登录到开发者电脑，具体步骤如下：

- 请选择并连接您机器人的 Wi-Fi 热点，密码为： `12345678`

- 通过 SSH 登录开发者电脑系统
  - 登录地址为：`10.192.1.4`
  - 登录密码为：`123456`
  - 在终端中输入以下命令（首次连接需确认命令）：

    代码块
    ```
    ssh guest@10.192.1.4
    ```

  - 开发者电脑系统配置：
    - 操作系统：Ubuntu 20.04.6 LTS
    - ROS2：系统默认安装ROS2 Foxy版本机器人系统
    - ROS1：系统默认安装ROS1 Noetic 版本机器人系统

2. 底层运动控制开发接口

2.1 C++ 运动控制开发接口

2.1.1 概述

底层运动控制开发接口提供统一的C++/Python API，兼容ROS1、ROS2及非ROS系统，实现运动控制算法的快速移植与部署。通过硬件抽象层和标准化通信协议，开发者可无缝切换仿真与真实硬件环境，显著降低多平台适配成本。

2.1.2 安装运动控制开发库

- Linux x86_64 环境

  代码块
  ```
  git clone https://github.com/limxdynamics/limxsdk-lowlevel.git
  pip install limxsdk-lowlevel/python3/amd64/limxsdk-*-py3-none-any.whl
  ```

- Linux aarch64 环境

  代码块
  ```
  git clone https://github.com/limxdynamics/limxsdk-lowlevel.git
  pip install limxsdk-lowlevel/python3/aarch64/limxsdk-*-py3-none-any.whl
  ```

- Windows 环境

代码块

```
1  git clone https://github.com/limxdynamics/limxsdk-lowlevel.git
2  pip install limxsdk-lowlevel/python3/win/limxsdk-*-py3-none-any.whl
```

2.1.3 getInstance 接口介绍

函数名	**getInstance**
函数原型	static Tron2* getInstance();
功能概述	获取 Tron2 机器人类单例实例的指针
参数	无
返回值	Tron2*，指向 Tron2 实例的指针
备注	使用了单例模式，确保 Tron2 类只有一个实例存在于程序中

代码示例	代码块
	`#include <thread>`
	
	`// 包含头文件`
	`#include "limxsdk/tron2.h"`
	
	`// 使用 limxsdk 命名空间`
	`using namespace limxsdk;`
	
	`int main(int argc, char *argv[]){`
	`  // 获取单例实例`
	`  Tron2* robot = Tron2::getInstance();`
	
	`  // 无限循环以保持程序运行`
	`  while (true)`
	`  {`
	`  // 休眠 1000 毫秒`
	`std::this_thread::sleep_for(std::chrono::milliseconds(1000));`
	`  }`
	
	`  return 0;`
	`}`

2.1.4 init 接口介绍

函数名	**init**
函数原型	bool init(const std::string& robot_ip_address = "127.0.0.1");
功能概述	**初始化**运动控制算法程序的通信运行环境，通常在主函数中调用其它接口之前调用，完成初始化工作。
参数	robot_ip_address：机器人的 IP 地址。对于仿真，通常设置为 "127.0.0.1"，而对于真实机器人，设置为 "10.192.1.2"。
返回值	如果初始化成功，则返回 true；否则返回 false。
备注	无

代码示例	代码块
	`#include <thread>`
	
	`// 包含头文件`
	`#include "limxsdk/tron2.h"`
	
	`// 使用 limxsdk 命名空间`
	`using namespace limxsdk;`
	
	`int main(int argc, char *argv[]){`
	`  // 获取单例实例`
	`  Tron2* robot = Tron2::getInstance();`
	
	`  // 默认机器人 IP 地址`
	`  std::string robot_ip = "127.0.0.1";`
	`  if (argc > 1)`
	`  {`
	`  // 如果提供了命令行参数，则使用命令行参数作为机器人 IP 地址`
	`  robot_ip = argv[1];`
	`  }`
	
	`  // 初始化运动控制算法程序的通信运行环境`
	`  if (!robot->init(robot_ip))`
	`  {`
	`  // 如果初始化失败，则退出程序`
	`  exit(1);`
	`  }`

```
  // 无限循环以保持程序运行
  while (true)
  {
    // 休眠 1000 毫秒
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
  }
  return 0;
}
```

2.1.5 getMotorNumber 接口介绍

函数名	**getMotorNumber**
函数原型	uint32_t getMotorNumber();
功能概述	获取机器人中的**电机数量**。
参数	无
返回值	返回一个无符号整数，表示机器人中的总电机数量。
备注	通常情况下，机器人双臂形态的电机数量为16个（包含头部电机）

代码示例	代码块
	`#include <thread>`
	
	`// 包含头文件`
	`#include "limxsdk/tron2.h"`
	
	`// 使用 limxsdk 命名空间`
	`using namespace limxsdk;`
	
	`int main(int argc, char *argv[]){`
	`  // 获取单例实例`
	`  Tron2* robot = Tron2::getInstance();`
	
	`  // 默认机器人 IP 地址`
	`  std::string robot_ip = "127.0.0.1";`
	`  if (argc > 1)`
	`  {`
	`  // 如果提供了命令行参数，则使用命令行参数作为机器人 IP 地址`
	`  robot_ip = argv[1];`

```
    }

    // 初始化运动控制算法程序的通信运行环境
    if (!robot->init(robot_ip))
    {
      // 如果初始化失败，则退出程序
      exit(1);
    }

    // 获取机器人中的电机数量
    uint32_t motor_num = robot->getMotorNumber();

    // 无限循环以保持程序运行
    while (true)
    {
      // 休眠 1000 毫秒
      std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    }
    return 0;
  }
```

2.1.6 subscribeImuData 接口介绍

函数名	**subscribeImuData**
函数原型	void subscribeImuData(std::function<void(const ImuDataConstPtr&)> cb);
功能概述	订阅机器人的 **IMU数据**，并在接收到新的 IMU 数据时调用指定的回调函数。
参数	cb: 用于处理新 IMU 数据的回调函数。
返回值	无

备注	ImuData 数据结构原型如下：
	代码块
	`/**`
	` * @struct ImuData`
	` *`
	` * @brief 表示基于传感器反馈的机器人 IMU 数据的结构体。`
	` *`
	` * 此结构体封装了 IMU 数据，包括加速度计、陀螺仪和四元数。`

```
 */
struct ImuData {
  uint64_t stamp; // 时间戳，以纳秒为单位，通常表示记录或生成此数据时的时间。
  float acc[3];   // 用于存储 IMU 加速度计数据，以跟踪沿三个轴（X、Y、Z）的线性加速度。
  float gyro[3];  // 用于存储 IMU 陀螺仪数据，以跟踪沿三个轴（X、Y、Z）的角速度或旋转速度。
  float quat[4];  // 用于存储 IMU 四元数数据，表示在三维空间中的方向（w、x、y、z）。
};

// 智能指针类型别名
typedef std::shared_ptr<ImuData> ImuDataPtr;
typedef std::shared_ptr<ImuData const> ImuDataConstPtr;
```

代码示例

代码块

```
#include <thread>

// 包含头文件
#include "limxsdk/tron2.h"

// 使用 limxsdk 命名空间
using namespace limxsdk;

int main(int argc, char *argv[]){
  // 获取 单例实例
  Tron2* robot = Tron2::getInstance();

  // 默认机器人 IP 地址
  std::string robot_ip = "127.0.0.1";
  if (argc > 1)
  {
    // 如果提供了命令行参数，则使用命令行参数作为机器人 IP 地址
    robot_ip = argv[1];
  }

  // 初始化运动控制算法程序的通信运行环境
  if (!robot->init(robot_ip))
  {
    // 如果初始化失败，则退出程序
    exit(1);
  }

  // 订阅机器人状态更新，并指定回调函数
```

```
    robot->subscribeImuData([&](const ImuDataConstPtr& msg) {
      // 在这里处理接收到的 ImuData 数据
      // 注意：回调函数会在收到ImuData时被调用
    });

    // 无限循环以保持程序运行
    while (true)
    {
      // 休眠 1000 毫秒

std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    }
    return 0;
  }
```

2.1.7 subscribeRobotState 接口介绍

函数名	**subscribeRobotState**
函数原型	void subscribeRobotState(std::function<void(const RobotStateConstPtr&)> cb);
功能概述	订阅接收关于**机器人状态**的更新。
参数	cb：回调函数，当接收到机器人状态更新时将被调用。回调函数参数指向 RobotState 对象的常量指针。
返回值	无

备注	• RobotState 数据结构原型如下：
	代码块
	`/**`
	` * @struct RobotState`
	` *`
	` * @brief 代表基于传感器反馈的机器人状态的结构体。`
	` *`
	` * 此结构封装了各种数据点，可用于监控和控制机器人，包括 IMU 数据（加速度计、陀螺仪、四元数）、输出扭矩、当前角度和速度等。`
	` */`
	`struct RobotState {`
	`  // 默认构造函数`
	`  RobotState() { }`
	`  // 带参数的构造函数，用于初始化向量大小为 motor_num 的 tau、q、dq 向量，初始值均为 0.0`

```
  RobotState(int motor_num)
  : tau(motor_num, 0.0)
  , q(motor_num, 0.0)
  , dq(motor_num, 0.0) { }
  uint64_t stamp;              // 时间戳，通常表示记录或生成这些数据的时间，以纳秒为单位
  std::vector<float> tau;      // 用于存储当前估计的输出扭矩（以牛顿米为单位）的向量
  std::vector<float> q;        // 用于存储当前角度（以弧度为单位）的向量
  std::vector<float> dq;       // 用于存储当前速度（以弧度每秒为单位）的向量
};

// 智能指针类型别名
typedef std::shared_ptr<RobotState> RobotStatePtr;
typedef std::shared_ptr<RobotState const> RobotStateConstPtr;
```

代码示例

代码块

```
#include <thread>

// 包含头文件
#include "limxsdk/tron2.h"

// 使用 limxsdk 命名空间
using namespace limxsdk;

int main(int argc, char *argv[]){
  // 获取单例实例
  Tron2* robot = Tron2::getInstance();

  // 默认机器人 IP 地址
  std::string robot_ip = "127.0.0.1";
  if (argc > 1)
  {
    // 如果提供了命令行参数，则使用命令行参数作为机器人 IP 地址
    robot_ip = argv[1];
  }

  // 初始化运动控制算法程序的通信运行环境
  if (!robot->init(robot_ip))
  {
    // 如果初始化失败，则退出程序
    exit(1);
  }
```

```

  // 订阅机器人状态更新，并指定回调函数
  robot->subscribeRobotState([&](const RobotStateConstPtr& msg) {
    // 在这里处理接收到的 RobotState 数据
    // 注意：回调函数会在收到机器人状态更新时被调用
  });

  // 无限循环以保持程序运行
  while (true)
  {
    // 休眠 1000 毫秒
  std::this_thread::sleep_for(std::chrono::milliseconds(1000));
  }
  return 0;
}
```

2.1.8 publishRobotCmd 接口介绍

函数名	**publishRobotCmd**
函数原型	bool publishRobotCmd(const RobotCmd& cmd);
功能概述	发布一个命令来 **控制机器人的动作**。
参数	cmd：表示所需机器人命令的 RobotCmd 对象。
返回值	无
备注	

代码示例	代码块
	`#include <thread>`
	
	`// 包含头文件`
	`#include "limxsdk/tron2.h"`
	
	`// 使用 limxsdk 命名空间`
	`using namespace limxsdk;`
	
	`int main(int argc, char *argv[]){`
	`  // 获取单例实例`

```
    Tron2* robot = Tron2::getInstance();

    // 默认机器人 IP 地址
    std::string robot_ip = "127.0.0.1";
    if (argc > 1)
    {
      // 如果提供了命令行参数，则使用命令行参数作为机器人 IP 地址
      robot_ip = argv[1];
    }

    // 初始化运动控制算法程序的通信运行环境
    if (!robot->init(robot_ip))
    {
      // 如果初始化失败，则退出程序
      exit(1);
    }

    // 获取机器人中的电机数量
    uint32_t motor_num = robot->getMotorNumber();

    // 创建一个包含机器人电机数量的 RobotCmd 对象
    RobotCmd cmd(motor_num);

    // 发布控制指令
    robot->publishRobotCmd(cmd);

    // 无限循环以保持程序运行
    while (true)
    {
      // 休眠 1000 毫秒
      std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    }
    return 0;
  }
```

2.1.9 subscribeSensorJoy 接口介绍

函数名	**subscribeSensorJoy**
函数原型	void subscribeSensorJoy(std::function<void(const SensorJoyConstPtr&)> cb);
功能概述	

	在真机部署中，该方法用于订阅来自机器人**遥控器的数据**。当机器人接收到遥控器的数据时，将会调用指定的回调函数，并传递包含遥控器数据的 SensorJoy 结构体常量指针给回调函数进行处理。
参数	cb: 表示回调函数，用于接收机器人遥控器的数据。回调函数的参数类型为 SensorJoyConstPtr，即指向 SensorJoy 结构体常量的共享指针。
返回值	无
备注	

代码示例	代码块
	``
	#include <thread>
	
	// 包含头文件
	#include "limxsdk/tron2.h"
	
	// 使用 limxsdk 命名空间
	using namespace limxsdk;
	
	int main(int argc, char *argv[]){
	  // 获取单例实例
	  Tron2* robot = Tron2::getInstance();
	
	  // 默认机器人 IP 地址
	  std::string robot_ip = "127.0.0.1";
	  if (argc > 1)
	  {
	  // 如果提供了命令行参数，则使用命令行参数作为机器人 IP 地址
	  robot_ip = argv[1];
	  }
	
	  // 初始化运动控制算法程序的通信运行环境
	  if (!robot->init(robot_ip))
	  {
	  // 如果初始化失败，则退出程序
	  exit(1);
	  }
	
	  // 订阅机器人遥控器数据
	  robot->subscribeSensorJoy([&](const limxsdk::SensorJoyConstPtr &joy) {
	  // L1 & R1 按下
	  if (joy->buttons[4] == 1 && joy->buttons[7] == 1)
	  {
	  // 在这里执行相关操作
	  }
	``
	代码块

```

      // 处理摇杆数据
      double axes_left_horizontal = joy->axes[0];
      double axes_left_vertical = joy->axes[1];
      double axes_right_horizontal = joy->axes[2];
      double axes_right_vertical = joy->axes[3];
    });

    // 无限循环以保持程序运行
    while (true)
    {
      // 休眠 1000 毫秒

std::this_thread::sleep_for(std::chrono::milliseconds(1000));

    }
    return 0;
  }
```

2.1.10 subscribeDiagnosticValue 接口介绍

函数名	**subscribeDiagnosticValue**
函数原型	void subscribeDiagnosticValue(std::function<void(const DiagnosticValueConstPtr&)> cb);
功能概述	在真机部署中，该方法用于**订阅机器人的诊断值和状态信息**。当机器人发出诊断值时，系统会调用指定的回调函数，并传递包含诊断值的 `DiagnosticValue` 结构体常量指针给回调函数进行处理。这可以帮助实时监控机器人的健康状态，并及时做出反应以处理可能的问题。
参数	cb: 用于接收机器人诊断值的回调函数，其参数类型为 DiagnosticValueConstPtr，即指向 DiagnosticValue 结构体常量的共享指针。DiagnosticValue 结构体包含了机器人诊断值的信息，包括时间戳、级别、名称、代码和消息字段。
返回值	无
备注	

代码示例	代码块
	`#include <thread>`
	`#include <iostream>`
	
	`// 包含头文件`

```
#include "limxsdk/tron2.h"

// 使用 limxsdk 命名空间
using namespace limxsdk;

int main(int argc, char *argv[]){
  // 获取单例实例
  Tron2* robot = Tron2::getInstance();

  // 默认机器人 IP 地址
  std::string robot_ip = "127.0.0.1";
  if (argc > 1)
  {
    // 如果提供了命令行参数，则使用命令行参数作为机器人 IP 地址
    robot_ip = argv[1];
  }

  // 初始化运动控制算法程序的通信运行环境
  if (!robot->init(robot_ip))
  {
    // 如果初始化失败，则退出程序
    exit(1);
  }

  // 订阅机器人诊断数据
  robot->subscribeDiagnosticValue([&](const DiagnosticValueConstPtr& msg) {
    // 在这里处理机器人诊断值
    // 例如，可以根据诊断值的级别和消息来采取相应的措施
    std::cout << "Diagnostic Value: " << msg->name << std::endl;
    std::cout << "Level: " << msg->level << std::endl;
    std::cout << "Code: " << msg->code << std::endl;
    std::cout << "Message: " << msg->message << std::endl;
  });

  // 无限循环以保持程序运行
  while (true)
  {
    // 休眠 1000 毫秒
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
  }
  return 0;
}
```

2.1.11 setRobotLightEffect 接口介绍

函数名	**setRobotLightEffect**
函数原型	bool setRobotLightEffect(int effect);
功能概述	在真机部署中，该方法用于**设置机器人的灯光效果**。
参数	effect: 一个整数，表示所需的机器人灯光效果，具体定义见 `Tron2::LightEffect` 枚举。
返回值	bool: 指示机器人灯光效果是否成功设置。

备注	Tron2::LightEffect 枚举定义：
	代码块
	`enum LightEffect : int {`
	`  STATIC_RED = 0,  // 静态红光`
	`  STATIC_GREEN,  // 静态绿光`
	`  STATIC_BLUE,  // 静态蓝光`
	`  STATIC_CYAN,  // 静态青光`
	`  STATIC_PURPLE,  // 静态紫光`
	`  STATIC_YELLOW,  // 静态黄光`
	`  STATIC_WHITE,  // 静态白光`
	`  LOW_FLASH_RED,  // 红光闪烁（慢闪）`
	`  LOW_FLASH_GREEN,  // 绿光闪烁（慢闪）`
	`  LOW_FLASH_BLUE,  // 蓝光闪烁（慢闪）`
	`  LOW_FLASH_CYAN,  // 青光闪烁（慢闪）`
	`  LOW_FLASH_PURPLE,  // 紫光闪烁（慢闪）`
	`  LOW_FLASH_YELLOW,  // 黄光闪烁（慢闪）`
	`  LOW_FLASH_WHITE,  // 白光闪烁（慢闪）`
	`  FAST_FLASH_RED,  // 红光闪烁（快速闪）`
	`  FAST_FLASH_GREEN,  // 绿光闪烁（快速闪）`
	`  FAST_FLASH_BLUE,  // 蓝光闪烁（快速闪）`
	`  FAST_FLASH_CYAN,  // 青光闪烁（快速闪）`
	`  FAST_FLASH_PURPLE,  // 紫光闪烁（快速闪）`
	`  FAST_FLASH_YELLOW,  // 黄光闪烁（快速闪）`
	`  FAST_FLASH_WHITE  // 白光闪烁（快速闪）`
	`};`

代码示例	代码块
	`#include <thread>`
	
	`// 包含头文件`

```
#include "limxsdk/tron2.h"

// 使用 limxsdk 命名空间
using namespace limxsdk;

int main(int argc, char *argv[]){
  // 获取单例实例
  Tron2* robot = Tron2::getInstance();

  // 默认机器人 IP 地址
  std::string robot_ip = "127.0.0.1";
  if (argc > 1)
  {
    // 如果提供了命令行参数，则使用命令行参数作为机器人 IP 地址
    robot_ip = argv[1];
  }

  // 初始化运动控制算法程序的通信运行环境
  if (!robot->init(robot_ip))
  {
    // 如果初始化失败，则退出程序
    exit(1);
  }

  // 设置机器人灯光效果为静态红光
  robot->setRobotLightEffect(limxsdk::Tron2::STATIC_RED);

  // 无限循环以保持程序运行
  while (true)
  {
    // 休眠 1000 毫秒

std::this_thread::sleep_for(std::chrono::milliseconds(1000));

  }
  return 0;
}
```

2.2 Python 运动控制开发接口

2.2.1 概述

提供与C++相同功能的Python运动控制算法开发接口，使得不熟悉C++编程语言的开发者能够使用Python进行运动控制算法的开发。

Python语言易于学习，具有简洁清晰的语法和丰富的第三方库，使开发者能够更快速地上手并迅速实现算法。通过Python接口，开发者可以利用Python的动态特性进行快速原型设计和实验验证，加速算法的迭代和优化过程。同时，Python的跨平台性和强大的生态系统支持，使得运动算法能够更广泛地应用于不同平台和环境。

此外，算法模型的快速部署到仿真和真机环境中也得益于Python的灵活性，开发者可以使用Python轻松地将算法模型集成到各种仿真平台和真实硬件中，实现快速迭代和验证算法的性能。

> 💡 以下示例均为在python 3.10.4环境中验证

2.2.2 安装运动控制开发库

- Linux x86_64 环境

代码块

```
1   git clone https://github.com/limxdynamics/limxsdk-lowlevel.git
2   pip install limxsdk-lowlevel/python3/amd64/limxsdk-*-py3-none-any.whl
```

- Linux aarch64 环境

代码块

```
1   git clone https://github.com/limxdynamics/limxsdk-lowlevel.git
2   pip install limxsdk-lowlevel/python3/aarch64/limxsdk-*-py3-none-any.whl
```

- Windows 环境

代码块

```
1   git clone https://github.com/limxdynamics/limxsdk-lowlevel.git
2   pip install limxsdk-lowlevel/python3/win/limxsdk-*-py3-none-any.whl
```

2.2.3 __init__ 接口介绍

函数名	**__init__**
函数原型	def __init__(self, robot_type: robot.RobotType)
功能概述	在初始化时，指定机器人的类型，并创建一个相应类型的本地机器人实例。

参数	robot_type：表示机器人类型的枚举值，类型为RobotType.Tron2。
返回值	无
备注	无

代码示例	代码块
	``python
	import sys
	import limxsdk.robot.Robot as Robot
	import limxsdk.robot.RobotType as RobotType
	
	if __name__ == '__main__':
	    # 创建一个类型为Tron2的Robot实例
	    robot = Robot(RobotType.Tron2)
	``
	代码块

2.2.4 init 接口介绍

函数名	**init**
函数原型	def init(self, robot_ip: str = "127.0.0.1")
功能概述	初始化运动控制算法程序的通信运行环境，通常在主函数中调用其它接口之前调用，完成初始化工作。
参数	robot_ip：机器人的 IP 地址。对于仿真，通常设置为 "127.0.0.1"，而对于真实机器人，设置为 "10.192.1.2"。
返回值	成功：返回True 失败：返回False
备注	无

代码示例	代码块
	``python
	import sys
	import limxsdk.robot.Robot as Robot
	import limxsdk.robot.RobotType as RobotType
	
	if __name__ == '__main__':
	    # 创建一个Robot实例
	    robot = Robot(RobotType.Tron2)
	
	    robot_ip = "10.192.1.2"
	    # 检查是否提供了命令行参数作为机器人IP
	``
	代码块

```
    if len(sys.argv) > 1:
        robot_ip = sys.argv[1]

    # 使用IP地址初始化机器人的通信运行环境
    if not robot.init(robot_ip):
        sys.exit()
```

2.2.5 getMotorNumber 接口介绍

函数名	**getMotorNumber**
函数原型	def getJointLimit(self, timeout: float = -1.0)
功能概述	获取机器人中的电机数量。
参数	无
返回值	返回一个无符号整数，表示机器人中的总电机数量。
备注	通常情况下，机器人双臂形态的电机数量为16个（包含头部电机）

代码示例	代码块
	`import sys`
	`import limxsdk.robot.Robot as Robot`
	`import limxsdk.robot.RobotType as RobotType`
	
	`if __name__ == '__main__':`
	`  # 创建一个 Robot 实例`
	`  robot = Robot(RobotType.Tron2)`
	
	`  robot_ip = "10.192.1.2"`
	`  # 检查是否提供了机器人 IP 的命令行参数`
	`  if len(sys.argv) > 1:`
	`  robot_ip = sys.argv[1]`
	
	`  # 使用 robot_ip 初始化机器人`
	`  if not robot.init(robot_ip):`
	`  sys.exit()`
	
	`  # 获取机器人中的电机数量`
	`  motor_number = robot.getMotorNumber()`

2.2.6 subscribeImuData 接口介绍

函数名	**subscribeImuData**
函数原型	def subscribeImuData(self, callback: Callable[[datatypes.ImuData], Any])
功能概述	订阅机器人的 IMU数据，并在接收到新的 IMU 数据时调用指定的回调函数。
参数	callback：用于处理新 IMU 数据的回调函数。
返回值	成功：返回True 失败：返回False
备注	datatypes.ImuData 数据结构原型如下： 代码块 1 import sys 2 3 class ImuData(object): 4 __slots__ = ['stamp','acc','gyro','quat'] 5 def __init__(self): 6 self.stamp = 0 # 时间戳，通常表示记录或生成这些数据的时间，以纳秒为单位 7 self.acc = [0. for x in range(0, 3)] # 用于存储 IMU（惯性测量单元）加速度计数据，用于跟踪三个轴上的线性加速度 8 self.gyro = [0. for x in range(0, 3)] # 用于存储 IMU 陀螺仪数据，用于跟踪角速度或旋转速度 9 self.quat = [0. for x in range(0, 4)] # 用于存储 IMU 四元数数据，表示在三维空间中的方向
代码示例	代码块 1 import sys 2 from functools import partial 3 import limxsdk.robot.Robot as Robot 4 import limxsdk.robot.RobotType as RobotType 5 import limxsdk.datatypes as datatypes 6 7 class RobotReceiver: 8 # 订阅机器人的 IMU数据 9 def imuDataCallback(self, imu: datatypes.ImuData): 10 print("\n------\nrobot_state:" + \ 11 "\n stamp: " + str(imu.stamp) + \ 12 "\n acc: " + str(imu.acc) + \

```
                "\n  gyro: " + str(imu.gyro) + \
                "\n  quat: " + str(imu.quat))

if __name__ == '__main__':
    # 创建一个 Robot 实例
    robot = Robot(RobotType.Tron2)

    robot_ip = "10.192.1.2"
    # 检查是否提供了机器人 IP 的命令行参数
    if len(sys.argv) > 1:
        robot_ip = sys.argv[1]

    # 使用 robot_ip 初始化机器人
    if not robot.init(robot_ip):
        sys.exit()

    # 创建一个 RobotReceiver 实例来处理回调
    receiver = RobotReceiver()

    # 创建回调函数的 partial 函数
    imuDataCallback = partial(receiver.imuDataCallback)

    # 订阅机器人IMU数据
    robot.subscribeImuData(imuDataCallback)

    # 休眠 1 秒，防止程序退出
    import time
    while True:
        time.sleep(1)
```

2.2.7 subscribeRobotState 接口介绍

函数名	**subscribeRobotState**
函数原型	def subscribeRobotState(self, callback: Callable[[datatypes.RobotState], Any])
功能概述	订阅接收关于机器人状态的更新。
参数	callback：回调函数，当接收到机器人状态更新时将被调用。回调函数参数指向 datatypes.RobotState 对象。
返回值	成功：返回True 失败：返回False

备注	

代码示例	代码块
	``python
	import sys
	from functools import partial
	import limxsdk.robot.Robot as Robot
	import limxsdk.robot.RobotType as RobotType
	import limxsdk.datatypes as datatypes
	
	class RobotReceiver:
	    # 用于接收机器人状态的回调函数
	    def robotStateCallback(self, robot_state: datatypes.RobotState):
	        print("\n------\nrobot_state:" + \
	              "\n stamp: " + str(robot_state.stamp) + \
	              "\n tau: " + str(robot_state.tau) + \
	              "\n q: " + str(robot_state.q) + \
	              "\n dq: " + str(robot_state.dq))
	
	if __name__ == '__main__':
	    # 创建一个 Robot 实例
	    robot = Robot(RobotType.Tron2)
	
	    robot_ip = "10.192.1.2"
	    # 检查是否提供了机器人 IP 的命令行参数
	    if len(sys.argv) > 1:
	        robot_ip = sys.argv[1]
	
	    # 使用 robot_ip 初始化机器人
	    if not robot.init(robot_ip):
	        sys.exit()
	
	    # 创建一个 RobotReceiver 实例来处理回调
	    receiver = RobotReceiver()
	
	    # 创建回调函数的 partial 函数
	    robotStateCallback = partial(receiver.robotStateCallback)
	
	    # 订阅机器人状态
	    robot.subscribeRobotState(robotStateCallback)
	
	    # 休眠 1 秒，防止程序退出
	    import time
	    while True:
	        time.sleep(1)
	``
	代码块

2.2.8 publishRobotCmd 接口介绍

函数名	**publishRobotCmd**
函数原型	def publishRobotCmd(self, cmd: datatypes.RobotCmd)
功能概述	发布一个命令来控制机器人的动作。
参数	cmd：表示所需机器人命令的 datatypes.RobotCmd 对象
返回值	成功：返回True 失败：返回False
备注	• 以下示例为控制机械臂直接运动到零位点。请先控制机械臂运动到初始位置等接近零位的姿态，以防止运行代码后砸伤机械臂 • 此处的kpkd值仅为接口调用示例，在实际业务中控制机械臂时请替换为您需要的kpkd值

代码示例	代码块
	import sys
	import time
	import limxsdk.robot.Rate as Rate
	import limxsdk.robot.Robot as Robot
	import limxsdk.robot.RobotType as RobotType
	import limxsdk.datatypes as datatypes
	
	if __name__ == '__main__':
	    # 创建一个 Robot 实例
	    robot = Robot(RobotType.Tron2)
	
	    # 对于仿真，通常设置为 "127.0.0.1"，而对于真实机器人，设置为 "10.192.1.2"
	    robot_ip = "10.192.1.2"
	    # 检查是否提供了机器人 IP 的命令行参数
	    if len(sys.argv) > 1:
	        robot_ip = sys.argv[1]
	
	    # 使用 robot_ip 初始化机器人
	    if not robot.init(robot_ip):
	        sys.exit()
	
	    # 获取电机数量信息
	    motor_number = robot.getMotorNumber()
	
	    # 主循环以连续发布机器人命令
	    rate = Rate(300) # 300Hz

```
    cmd_msg = datatypes.RobotCmd()
    while True:
        # 设置时间戳、控制模式、关节位置、速度、力矩、Kp 和 Kd 的默认值
        # motor_names 对应您要控制的关节名称
        # 注意以下为格式示例，在实际使用过程中应填充具体参数
        cmd_msg.stamp = time.time_ns()
        cmd_msg.mode = [0.0 for _ in range(16)] # 实际使用无需改变
        cmd_msg.q = [0.0 for _ in range(16)]    # 填充您规划的角度，控制频率300hz
        cmd_msg.dq = [0.0 for _ in range(16)]   # 实际使用无需改变
        cmd_msg.tau = [0.0 for _ in range(16)]  # 实际使用无需改变
        cmd_msg.Kp = [21,21,15,15,10,10,10,21,21,15,15,10,10,10,10,10]
        cmd_msg.Kd = [0.6,0.6,0.75,0.75,0.5,0.5,0.5,0.6,0.6,0.75,0.75,0.5,0.5,0.5,0.5,0.5]
        cmd_msg.motor_names = ["" for _ in range(16)]   # 实际使用无需改变
        robot.publishRobotCmd(cmd_msg)  # 发布机器人命令
        rate.sleep()  # 控制循环频率

```

2.2.9 subscribeSensorJoy 接口介绍

函数名	**subscribeSensorJoy**
函数原型	def subscribeSensorJoy(self, callback: Callable[[datatypes.SensorJoy], Any])
功能概述	在真机部署中，该方法用于订阅来自机器人遥控器的数据。当机器人接收到遥控器的数据时，将会调用指定的回调函数，并传递包含遥控器数据的 datatypes.SensorJoy 结构体常量指针给回调函数进行处理。
参数	callback: 表示回调函数，用于接收机器人遥控器的数据。回调函数的参数类型为 datatypes.SensorJoy
返回值	成功：返回True 失败：返回False
备注	

代码示例

```
import sys
import time
from functools import partial
import limxsdk.robot.Robot as Robot
import limxsdk.robot.RobotType as RobotType
import limxsdk.datatypes as datatypes

class RobotReceiver:
    # 用于接收遥控器数据的回调函数
    def sensorJoyCallback(self, sensor_joy: datatypes.SensorJoy):
        print("\n------\nsensor_joy:" + \
              "\n  stamp: " + str(sensor_joy.stamp) + \
              "\n  axes: " + str(sensor_joy.axes) + \
              "\n  buttons: " + str(sensor_joy.buttons))

if __name__ == '__main__':
    # 创建一个 Robot 实例
    robot = Robot(RobotType.Tron2)

    robot_ip = "10.192.1.2"
    # 检查是否提供了机器人 IP 的命令行参数
    if len(sys.argv) > 1:
        robot_ip = sys.argv[1]

    # 使用 robot_ip 初始化机器人
    if not robot.init(robot_ip):
        sys.exit()

    # 创建一个 RobotReceiver 实例来处理回调
    receiver = RobotReceiver()

    # 创建回调函数的 partial 函数
    sensorJoyCallback = partial(receiver.sensorJoyCallback)

    # 订阅机器人遥控数据
    robot.subscribeSensorJoy(sensorJoyCallback)

    # 休眠 1 秒，防止程序退出
    import time
    while True:
        time.sleep(1)

```

2.2.10 subscribeDiagnosticValue 接口介绍

函数名	**subscribeDiagnosticValue**
函数原型	def subscribeDiagnosticValue(self, callback: Callable[[datatypes.DiagnosticValue], Any])
功能概述	在真机部署中，该方法用于订阅机器人的诊断值和状态信息。当机器人发出诊断值时，系统会调用指定的回调函数，并传递包含诊断值的 datatypes.DiagnosticValue 结构对象给回调函数进行处理。这可以帮助实时监控机器人的健康状态，并及时做出反应以处理可能的问题。
参数	callback: 用于接收机器人诊断值的回调函数，其参数类型为 datatypes.DiagnosticValue。datatypes.DiagnosticValue 结构体包含了机器人诊断值的信息，包括时间戳、级别、名称、代码和消息字段。
返回值	成功：返回True 失败：返回False
备注	
代码示例	代码块

```python
import sys
from functools import partial
import limxsdk.robot.Robot as Robot
import limxsdk.robot.RobotType as RobotType
import limxsdk.datatypes as datatypes

class RobotReceiver:
    # 用于接收诊断值的回调函数
    def diagnosticValueCallback(self, diagnostic_value: datatypes.DiagnosticValue):
        print("\n------\ndiagnostic_value:" + \
              "\n  stamp: " + str(diagnostic_value.stamp) + \
              "\n  name: " + diagnostic_value.name + \
              "\n  level: " + str(diagnostic_value.level) + \
              "\n  code: " + str(diagnostic_value.code) + \
              "\n  message: " + diagnostic_value.message)

if __name__ == '__main__':
    # 创建一个 Robot 实例
    robot = Robot(RobotType.Tron2)

    robot_ip = "10.192.1.2"
    # 检查是否提供了机器人 IP 的命令行参数
    if len(sys.argv) > 1:
        robot_ip = sys.argv[1]

    # 使用 robot_ip 初始化机器人
```

```
    if not robot.init(robot_ip):
        sys.exit()

    # 创建一个 RobotReceiver 实例来处理回调
    receiver = RobotReceiver()

    # 创建回调函数的 partial 函数
    diagnosticValueCallback = partial(receiver.diagnosticValueCallback)

    # 订阅机器人诊断信息
    robot.subscribeDiagnosticValue(diagnosticValueCallback)

    # 休眠 1 秒，防止程序退出
    import time
    while True:
        time.sleep(1)

```

2.2.11 setRobotLightEffect 接口介绍

函数名	**setRobotLightEffect**
函数原型	def setRobotLightEffect(self, effect: datatypes.LightEffect)
功能概述	在真机部署中，该方法用于设置机器人的灯光效果。
参数	effect: 一个整数，表示所需的机器人灯光效果，具体定义见 `Tron2::LightEffect` 枚举。
返回值	成功：返回True 失败：返回False
备注	

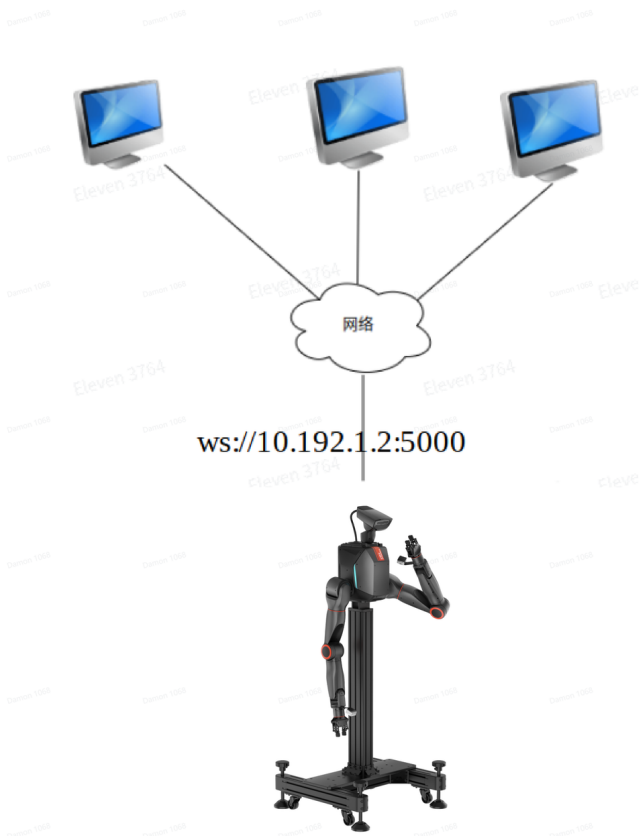
代码示例	代码块
	import sys
	from functools import partial
	import limxsdk.robot.Robot as Robot
	import limxsdk.robot.RobotType as RobotType
	import limxsdk.datatypes as datatypes
	
	class RobotReceiver:
	
	if __name__ == '__main__':

```
    # 创建一个 Robot 实例
    robot = Robot(RobotType.Tron2)

    robot_ip = "10.192.1.2"
    # 检查是否提供了机器人 IP 的命令行参数
    if len(sys.argv) > 1:
        robot_ip = sys.argv[1]

    # 使用 robot_ip 初始化机器人
    if not robot.init(robot_ip):
        sys.exit()

    # 设置机器人灯光效果为静态红光
robot.setRobotLightEffect(datatypes.LightEffect.STATIC_RED)

```

3. 上层应用开发接口

3.1 概述

在 `开发者模式` 下机器人通过WebSocket通信端口 `5000` 来接收用户端请求指令。WebSocket是一种实时通信协议，在机器人和用户端之间建立长连接，以便快速有效地传输控制信息和数据。如下图所示：



3.2 通信协议格式

当机器人通过 WebSocket 接收客户端指令时，采用 JSON 数据协议进行信息传递。这种方式具有显著优势：WebSocket 是一种全双工通信协议，能够在客户端与服务器之间建立实时、低延迟的连接，特别适合频繁交互的应用场景。JSON 数据协议则以其简洁、可读性强的结构，确保数据传输直观明了，且具有跨平台、跨语言的兼容性。WebSocket 与 JSON 的结合不仅与编程语言无关，适用于各种设备和系统，还能提升开发的灵活性和维护的便利性。

- 请求数据格式包含以下字段：
  - `accid`：机器人序列号，注意修为您机器人的序列号；
  - `title`：指令名称，以“`request_`”为前缀；
  - `timestamp`：指令发出时间戳，单位为毫秒；
  - `guid`：指令的唯一标识符，用于区分不同的请求指令。如果是同步接口，则需要在“response_xxx”响应消息中通过guid字段将值带回给客户端。客户端接收到响应消息后，可以通过比较guid字段的值是否与请求指令中的值相同来判断指令是否执行完成；
  - `data`：存放请求指令的数据内容。可以根据具体需求包含多个子字段，以存放请求指令所需的数据内容，例如执行动作的参数、发送消息的文本内容等等；
- 示例如下：

代码块

```
{
  "accid": "DA_TRON2A_001", # 机器人序列号，注意修为您机器人的序列号
  "title": "request_xxx",   # 指令名称，以“request_”为前缀
  "timestamp": 1672373633989, # 指令发出时间戳，单位为毫秒
  "guid": "746d937cd8094f6a98c9577aaf213d98", # 指令的唯一标识符，用于区分不同的请求指令
  "data": {}  # 存放请求指令的数据内容
}
```

- 响应数据格式包含以下字段：
  - `accid`：机器人序列号，注意修为您机器人的序列号；
  - `title`：指令名称，以“`response_`”为前缀；
  - `timestamp`：指令发出时间戳，单位为毫秒；
  - `guid`：与对应请求指令的`guid`值相同；
  - `data`：至少应该包含一个“result”子字段，用于存放请求指令的执行结果数据。如果有需要，还可以包含其他子字段，例如错误码、错误信息等用于描述操作结果的信息；
- 示例如下：

```
代码块
  "accid": "DA_TRON2A_001",   # 机器人序列号，注意修为您机器人的序列号
  "title": "response_xxx",  # 指令名称，以“response_”为前缀
  "timestamp": 1672373633989, # 指令发出时间戳，单位为毫秒
  "guid": "746d937cd8094f6a98c9577aaf213d98", # 与对应请求指令的guid值相同
  "data": { # 存放响应指令的具体数据内容
    "result": "success"  # “result” 用于存放请求指令处理是否成功，它的值为：“success 或 fail_xxx”
  }
}
```

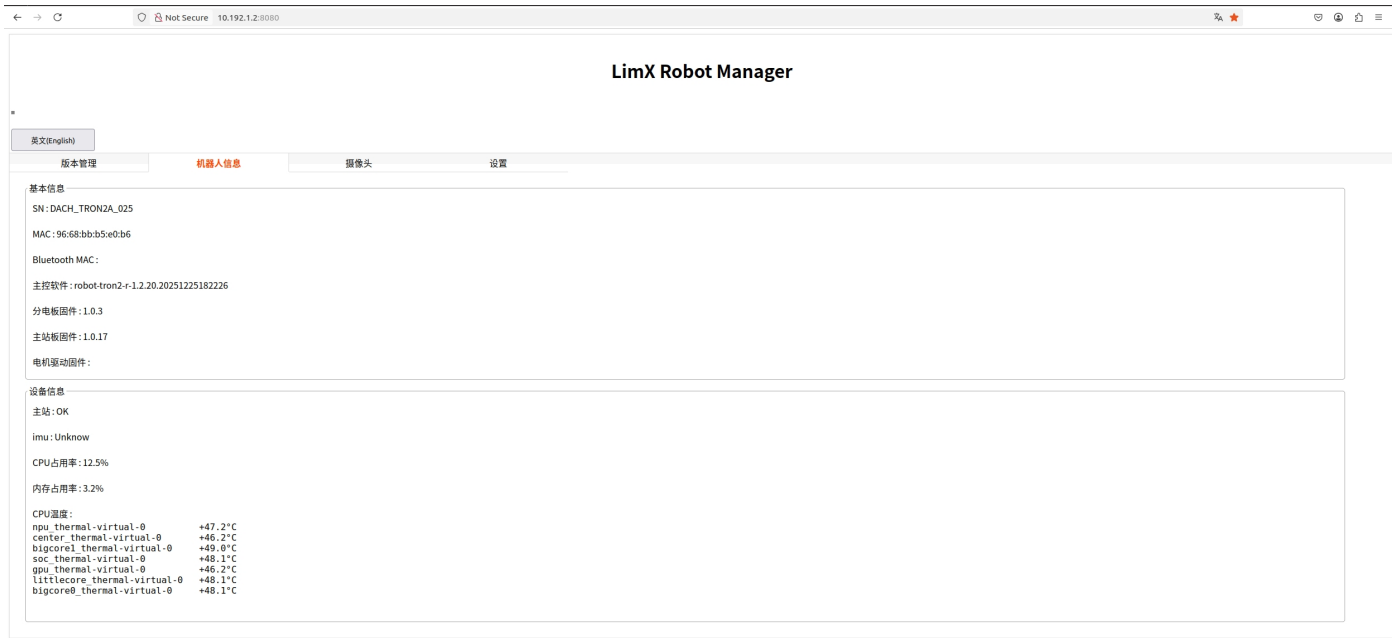
- 消息推送：它是机器人主动向客户端发送信息的过程。这些信息可以包括机器人的序列号、当前运行状态、执行的操作等数据。通过及时地向客户端发送这些信息，机器人可以帮助客户端更好地理解它的工作状态，从而更好地使用它提供的服务。它的数据格式包含以下字段：
  - `accid`：机器人序列号，注意修为您机器人的序列号；
  - `title`：指令名称，以“`notify_`”为前缀；
  - `timestamp`：消息发出时间戳，单位为毫秒；
  - `guid`：消息的guid值，唯一标识这条消息；
  - `data`：存放消息数据内容。可以根据具体需求包含多个子字段，以存放请求指令所需的数据内容；
- 示例如下：

```
代码块
{
  "accid": "DA_TRON2A_001",   # 机器人序列号，注意修为您机器人的序列号
  "title": "notify_xxx",  # 消息名称，以“notify_”为前缀
  "timestamp": 1672373633989, # 消息发出时间戳，单位为毫秒
  "guid": "746d937cd8094f6a98c9577aaf213d98", # 消息的guid值，唯一标识这条消息
  "data": { } # 存放消息数据内容
}
```

3.3 查看软件序列号(ACCID)

- 连接机器人无线网络
  - 机器人开机完成后，使用个人电脑连接机器人Wi-Fi，名称格式通常为「DA_TRON2A_xxx」
  - 输入Wi-Fi密码：`12345678`

- 在浏览器中输入 `http://10.192.1.2:8080` 可以进入“机器人信息页”，并查看机器人信息。如下图所示，页面中显示的SN (序列号) 为DA_TRON2A_001，其中DA_TRON2A_001便是此机器人的软件序列号。



3.4 通信测试方法

Postman是一个流行的API开发环境，可以用于测试WebSocket接口。使用Postman测试WebSocket接口，请按照以下步骤操作：

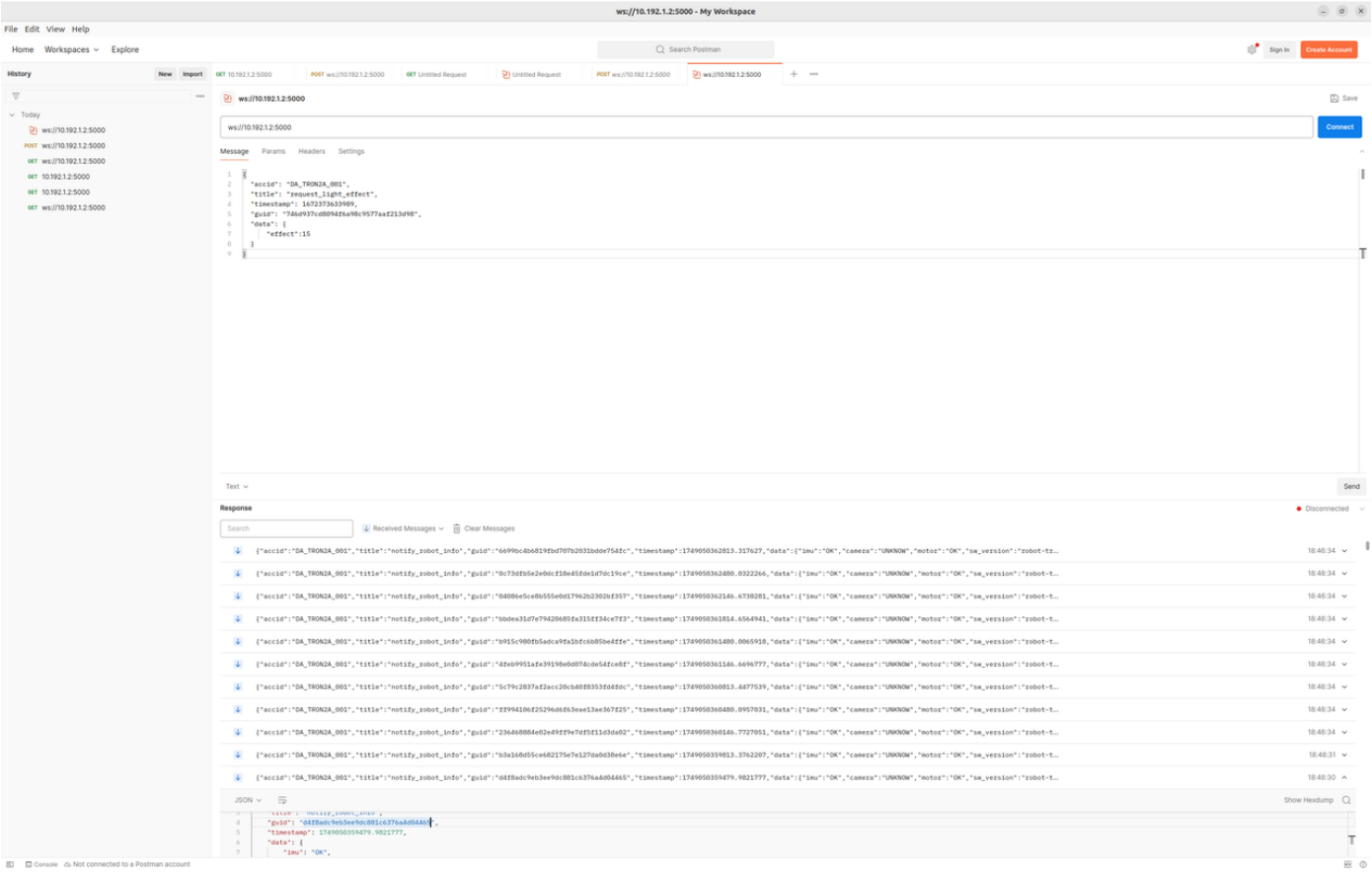
1. 安装postman客户端，下载地址：https://www.postman.com/downloads/?utm_source=postman-home；
   - windows环境下，可直接点击exe文件进行安装
   - ubuntu环境下，可使用以下命令通过snap进行安装和卸载

代码块

```
#安装指令
sudo apt update
sudo snap install postman
#卸载指令
sudo snap remove postman
```

2. 打开Postman，并创建一个WebSocket的请求；
3. 连接机器人无线网络；
   - 机器人开机完成后，使用个人电脑连接机器人Wi-Fi
   - 输入Wi-Fi密码：`12345678`
4. 在请求的URL中输入WebSocket接口的地址，例如“ws://10.192.1.2:5000”；

5. 在“Message”中，输入要发送的指令请求；
6. 单击“Send”按钮，发送请求指令；
7. 发送指令后，可以从服务器接收响应消息。使用Postman的响应窗口查看服务器返回的数据，并检查是否符合预期结果。



3.5 协议接口定义

机器人接口设计遵循与遥控器操控一致的流程和状态流转，确保调用顺序、响应时序及状态过渡与遥控器控制逻辑严格对齐。用户通过接口调用可获得如同使用遥控器的直观体验，同时支持遥控器与接口间的无缝切换，实现统一、稳定的机器人操控效果。

3.5.1 MoveJ接口介绍

3.5.1.1 请求：request_movej

💡 注意：

- moveJ调用时会判断每个关节的关节角是否到达限位，如果超过限位，则会报错超限，本次调用不执行

  moveJ关节限位（按照左臂关节上到下->右臂关节上到下的顺序）：

  上限：[2.5994, 2.9671, 1.4835, 0.5236, 1.3963, 0.7854, 1.5708, 2.5994, 2.9671, 1.4835, 0.5236, 1.3963, 0.7854, 1.5708]

下限：[-3.1416, -0.2618, -3.6652, -2.618, -1.7453, -0.7854, -1.5708, -3.1416, -0.2618, -3.6652, -2.618, -1.7453, -0.7854, -1.5708]

- 只需要输入14个关节值

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号;
  "title": "request_movej",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "time": 5, // 5秒运动到指定位置
    "joint": [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]  //14个joint值，单位rad
  }
}
```

示例：

代码块

```
{
    "accid": "DACH_TRON2A_026",
    "title": "request_movej",
    "timestamp": 1672373633989,
    "guid": "746d937cd8094f6a98c9577aaf213d98",
    "data": {
        "time": 5,
        "joint": [0,0,0,-1.6,0,0,0,0,0,0,-1.6,0,0,0]
    }
}
```

3.5.1.2 响应：response_movej

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号;
  "title": "response_movej",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
```

```
    "result": "success"  // success: 成功, fail_invalid_cmd:错误的命令(缺少字段)
  }
}
```

3.5.2 MoveP接口介绍

3.5.2.1 请求：request_movep

⚠ 注意：

- moveP调用时会判断左右手的末端位置是否在可达范围内，如果超过限位，则会报错超限，本次调用不执行

  moveP关节限位（按照x_min, x_max, y_min, y_max, z_min, z_max，单位m）

  左臂：[[0.250, 0.732], [-0.213, 0.900], [-0.673, 0.5]]

  右臂：[[0.250, 0.732], [-0.900, 0.213], [-0.673, 0.5]]

- 旋转姿态没有限制，但建议rpy欧拉角保持在[-90°, 90°]之间，如果只需控制位置，建议姿态保持3*3单位矩阵

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号;
  "title": "request_movep",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "time": 5, //5秒运动到该位置
    "pos": []  //left_position(x,y,z) + 空间旋转矩阵matrix 3*3 + right_position(x,y,z)+ 空间旋转矩阵matrix 3*3 ,  unit: m, rad.
  }
}
```

> 示例：

代码块

```
{
  "accid": "DACH_TRON2A_026", // 机器人序列号，注意修为您机器人的序列号;
  "title": "request_movep",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
```

```
    "data": {
      "time": 5, //5秒运动到该位置
      "pos": [0.3,0.2,-0.3,1,0,0,0,1,0,0,0,1,0.3,-0.2,-0.3,1,0,0,0,1,0,0,0,1]
    }
  }
```

3.5.2.2 响应：response_movep

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号;
  "title": "response_movep",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "result": "success"  // success: 成功, fail_invalid_cmd:错误的命令(缺少字段)
  }
}
```

3.5.3 ServoJ 控制指令

3.5.3.1 请求：request_servoj

- 推荐在实时系统中按控制频率 >= 500Hz 要求来控制机械臂运动，以保证控制效果和稳定性。

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号;
  "title": "request_servoj",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "q": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    "v": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    "kp": [100.0, 110.0, 120.0, 130.0, 140.0, 150.0, 160.0, 170.0, 180.0, 190.0, 200.0, 210.0, 220.0, 230.0],
    "kd": [10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0, 17.0, 18.0, 19.0, 20.0, 21.0, 22.0, 23.0],
    "tau": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    "mode": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    "na": 0
```

```
    }
  }

  // 请注意将以上参数更换为实际参数
```

3.5.3.2 响应：无

3.5.3.3 消息推送：notify_servoJ

当 ServoJ 控制操作执行失败时，服务器会主动推送此消息，告知客户端失败原因。

代码块

```
{
  "accid": "DACH_TRON2A_001",
  "title": "notify_servoJ",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "result": "fail_invalid_cmd"  # fail_invalid_cmd: 非法指令， fail_motor: 电机错误
  }
}
```

3.5.4 获取机器人关节状态

3.5.4.1 请求：request_get_joint_state

此请求用于获取机器人的各个关节状态。

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号；
  "title": "request_get_joint_state",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {}
}
```

3.5.4.2 响应：response_get_joint_state

接收到请求后，返回机器人各个关节当前状态。

代码块

```
{
  "accid": "DACH_TRON2A_001",
  "title": "response_get_joint_state",
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "timestamp": 1672373633989,
  "data": {
      "result": "success"  # fail_not_data
      "timestamp": 0,
      "names": [], # 各个关节的名称
      "q": [],     # 各个关节的位置
      "dq": [],    # 各个关节的速度
      "tau": [],   # 各个关节的扭矩
  }
}
```

3.5.5 获取双臂末端位姿

3.5.5.1 请求：request_get_move_pose

通过此接口获取机器人双臂的末端位姿信息。

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号；
  "title": "request_get_move_pose",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {}
}
```

3.5.5.2 响应：response_get_move_pose

接收到请求后，返回双臂当前位姿的相关信息。

代码块

```
{
  "accid": "DACH_TRON2A_001",
  "title": "response_get_move_pose",
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "timestamp": 1672373633989,
  "data": {
      "left_position": [0.0, 0.0, 0.0], # 表示左臂末端的位置，单位为米，顺序为 x, y, z
```

```
        "left_quat": [0.0, 0.0, 0.0, 1.0], # 表示左臂末端的姿态，以四元数表示，顺序为 w, x, y, z
        "right_position": [0.0, 0.0, 0.0], # 表示右臂末端的位置，单位为米，顺序为 x, y, z
        "right_quat": [0.0, 0.0, 0.0, 1.0], # 表示右臂末端的姿态，以四元数表示，顺序为 w, x, y, z
        "timestamp": 1672373633989, # 表示数据时戳，单位为毫秒
        "result": "success"  # fail_not_data
    }
}
```

3.5.5.3 消息推送：无

3.5.6 紧急停止

3.5.6.1 请求：request_emgy_stop

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号；
  "title": "request_emgy_stop",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {}
}
```

3.5.6.2 响应：response_emgy_stop

代码块

```
{
  "accid": "DACH_TRON2A_001",
  "title": "response_emgy_stop",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "result": "success"  // success: 成功, fail_imu: IMU 错误, fail_motor: 电机错误
  }
}
```

3.5.6.3 消息推送：无

3.5.7 设置灯效

3.5.7.1 请求：request_light_effect

该接口用于设置机器人的灯光效果。用户可以通过此接口向机器人发送特定的灯光效果指令，机器人会根据指令调整其灯光显示。

代码块

```
{
  "accid": "DACH_TRON2A_001", // 机器人序列号，注意修为您机器人的序列号；
  "title": "request_light_effect",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
      "effect": 1
  }
}
```

请求参数说明：

参数名	类型	描述
data.effect	数字	灯光效果的编号，对应不同的灯光显示模式，具体映射关系如下： 1: STATIC_RED（静态红色） 2: STATIC_GREEN（静态绿色） 3: STATIC_BLUE（静态蓝色） 4: STATIC_CYAN（静态青色） 5: STATIC_PURPLE（静态紫色） 6: STATIC_YELLOW（静态黄色） 7: STATIC_WHITE（静态白色） 8: LOW_FLASH_RED（低频闪烁红色） 9: LOW_FLASH_GREEN（低频闪烁绿色） 10: LOW_FLASH_BLUE（低频闪烁蓝色） 11: LOW_FLASH_CYAN（低频闪烁青色） 12: LOW_FLASH_PURPLE（低频闪烁紫色） 13: LOW_FLASH_YELLOW（低频闪烁黄色） 14: LOW_FLASH_WHITE（低频闪烁白色） 15: FAST_FLASH_RED（高频闪烁红色） 16: FAST_FLASH_GREEN（高频闪烁绿色） 17: FAST_FLASH_BLUE（高频闪烁蓝色） 18: FAST_FLASH_CYAN（高频闪烁青色） 19: FAST_FLASH_PURPLE（高频闪烁紫色）

		20: FAST_FLASH_YELLOW（高频闪烁黄色） 21: FAST_FLASH_WHITE（高频闪烁白色）

3.5.7.2 响应：response_light_effect

代码块

```
{
  "accid": "DACH_TRON2A_001",
  "title": "response_light_effect",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "effect": 1,
    "result": "success"  // success: 成功, fail_light_effect: 失败
  }
}
```

3.5.7.3 消息推送：无

3.5.8 全局消息

3.5.8.1 机器人基本信息

机器人基本信息每秒上报一次，包含以下内容：

- `accid`：机器人序列号
- `title`：notify_robot_info
- `timestamp`：消息发出时间戳，单位为毫秒
- `guid`：消息的guid值，唯一标识这条消息
- `data`：存放消息内容，示例如下：

代码块

```
{
  "accid": "DACH_TRON2A_001",   # 机器人序列号
  "title": "notify_robot_info",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "imu": "OK",    # 机器人IMU诊断信息
    "camera": "OK", # 机器人相机诊断信息
    "motor": "OK",  # 机器人电机诊断信息
```

```
      "sw_version": "robot-tron2-2.0.10.20241111103012", # 机器人本体软件版本信息
      "battery": 95,  # 机器人电量
      "status": "UNKNOW" # 保留字段
    }
  }
```

3.5.8.2 非法指令消息

当机器人收到非法格式的请求指令时，发送此消息，包含以下内容：

- `accid`：机器人序列号
- `title`：notify_invalid_request
- `timestamp`：消息发出时间戳，单位为毫秒
- `guid`：消息的guid值，唯一标识这条消息
- `data`：存放消息内容
- 示例如下：

代码块

```
{
  "accid": "DA_TRON2A_001",
  "title": "notify_invalid_request",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": "返回原请求指令内容，便于客户端排查问题"
}
```

3.6 协议接口调用示例

3.6.1 C++ 示例实现

- 安装依赖：以Ubuntu 20.04系统为例，安装websocketpp、nlohmann/json和boost依赖：

代码块

```
sudo apt-get install libboost-all-dev libwebsocketpp-dev nlohmann-json3-dev
```

- 编译代码

代码块

```
g++ -std=c++11 -o websocket_client websocket_client.cpp -lssl -lcrypto -lboost_system -lpthread
```

- 运行程序

代码块

```
./websocket_client
```

- websocket_client.cpp 实现

代码块

```
#include <iostream>
#include <atomic>
#include <string>
#include <thread>
#include <chrono>
#include <websocketpp/client.hpp>
#include <websocketpp/config/asio.hpp>
#include <nlohmann/json.hpp>
#include <boost/uuid/uuid.hpp>
#include <boost/uuid/uuid_generators.hpp>
#include <boost/uuid/uuid_io.hpp>

using json = nlohmann::json;
using websocketpp::client;
using websocketpp::connection_hdl;

// Replace this value with the actual serial number (SN) of the robot.
static std::string ACCID = "";

// Replace it with the real IP address of the robot.
// Usually, for simulation, it is: 127.0.0.1
// for a real machine, it is: 10.192.1.2
const std::string ROBOT_IP = "10.192.1.2";

// WebSocket client instance
static client<websocketpp::config::asio> ws_client;

// Atomic flag for graceful exit
static std::atomic<bool> should_exit(false);

// Connection handle for sending messages
static connection_hdl current_hdl;
```

```

// Generate dynamic GUID
static std::string generate_guid()
{
  boost::uuids::random_generator gen;
  boost::uuids::uuid u = gen();
  return boost::uuids::to_string(u);
}

// Send WebSocket request with title and data
static void send_request(const std::string &title, const json &data = json::object())
{
  json message;

  // Adding necessary fields to the message
  message["accid"] = ACCID;
  message["title"] = title;
  message["timestamp"] = std::chrono::duration_cast<std::chrono::milliseconds>(
                                std::chrono::system_clock::now().time_since_epoch())
                                .count();
  message["guid"] = generate_guid();
  message["data"] = data;

  std::string message_str = message.dump();

  // Send the message through WebSocket
  ws_client.send(current_hdl, message_str, websocketpp::frame::opcode::text);
}

// Handle user commands
void handle_commands()
{
  std::cout << "Enter command ('movej', 'movep', 'light', 'stop') or 'exit' to quit:\n";
  while (!should_exit)
  {
    std::string command;
    std::cin >> command;

    if (command == "exit")
    {
      should_exit = true;
      return;
```

```
    }
    else if (command == "movej")
    {
      nlohmann::json data = {
          {"joint", {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ,0}},
          {"time", 5}};
      send_request("request_movej", data);
    }
    else if (command == "movep")
    {
      nlohmann::json data = {
          {"pos", {0.3, 0.2, -0.3, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0.3, -0.2, -0.3, 1, 0, 0, 0, 1, 0, 0, 0, 1}},
          {"time", 5}};
      send_request("request_movep", data);
    }
    else if (command == "light")
    {
      send_request("request_light_effect", {{"effect", 1}});
    }
    else if (command == "stop")
    {
      send_request("request_emgy_stop"); // Send emergency stop request
    }

    sleep(1);

    std::cout << "Enter command ('movej', 'movep', 'light', 'stop') or 'exit' to quit:\n";
  }
}

// WebSocket open callback
static void on_open(connection_hdl hdl)
{
  std::cout << "Connected!" << std::endl;

  // Save connection handle for sending messages later
  current_hdl = hdl;

  // Start handling commands in a separate thread
  std::thread(handle_commands).detach();
}

// WebSocket TCP initialization handler
static void on_tcp_init(connection_hdl hdl)
{
```

```
  auto con = ws_client.get_con_from_hdl(hdl);

  // Obtain the underlying TCP socket
  auto &socket = con->get_socket().lowest_layer();

  // Configure socket options
  try
  {
    boost::system::error_code ec;

    // Set send buffer size (e.g., 2MB)
    const size_t sendBufferSize = 2 * 1024 * 1024;

socket.set_option(websocketpp::lib::asio::socket_base::send_buffer_size(sendBufferSize), ec);

    if (ec)
    {
      printf("Failed to set send buffer size: %s", ec.message().c_str());
    }

    // Set receive buffer size (e.g., 2MB)
    const size_t recvBufferSize = 2 * 1024 * 1024;

socket.set_option(websocketpp::lib::asio::socket_base::receive_buffer_size(recvBufferSize), ec);

    if (ec)
    {
      printf("Failed to set receive buffer size: %s", ec.message().c_str());
    }

    // Disable Nagle's algorithm to reduce latency
    socket.set_option(websocketpp::lib::asio::ip::tcp::no_delay(true), ec);

    if (ec)
    {
      printf("Failed to disable Nagle's algorithm: %s", ec.message().c_str());
    }
  }
  catch (const std::exception &e)
  {
    printf("Socket configuration exception: %s", e.what());
  }
}

```

```
// WebSocket message callback
static void on_message(connection_hdl hdl, client<websocketpp::config::asio>::message_ptr msg)
{
  // Parse JSON data from message payload
  json data = json::parse(msg->get_payload());

  // Extract 'accid' field if present
  if (data.contains("accid") && data["accid"].is_string() && ACCID.empty())
  {
    ACCID = data["accid"].get<std::string>();
  }

  if (msg->get_payload().find("notify_robot_info") == std::string::npos)
  {
    std::cout << "Received message: " << msg->get_payload() << std::endl;
  }
}

// WebSocket close callback
static void on_close(connection_hdl hdl)
{
  std::cout << "Connection closed." << std::endl;
}

// Close WebSocket connection
static void close_connection(connection_hdl hdl)
{
  ws_client.close(hdl, websocketpp::close::status::normal, "Normal closure"); // Close connection normally
}

int main()
{
  ws_client.init_asio(); // Initialize ASIO for WebSocket client

  ws_client.set_access_channels(websocketpp::log::alevel::none);

  // Set WebSocket event handlers
  ws_client.set_open_handler(&on_open);         // Set open handler
  ws_client.set_message_handler(&on_message);   // Set message handler
  ws_client.set_close_handler(&on_close);       // Set close handler
  ws_client.set_tcp_init_handler(&on_tcp_init); // Set tcp init handler

  std::string server_uri = "ws://" + ROBOT_IP + ":5000"; // WebSocket server URI

```

```
  websocketpp::lib::error_code ec;
  client<websocketpp::config::asio>::connection_ptr con = ws_client.get_connection(server_uri, ec); // Get connection pointer

  if (ec)
  {
    std::cout << "Error: " << ec.message() << std::endl;
    return 1; // Exit if connection error occurs
  }

  connection_hdl hdl = con->get_handle(); // Get connection handle
  ws_client.connect(con);                 // Connect to server
  std::cout << "Press Ctrl+C to exit." << std::endl;

  // Run the WebSocket client loop
  ws_client.run();

  return 0;
}
```

3.6.2 Python 示例实现

- 环境准备：以Ubuntu 22.04系统,python3.10.4版本为例，安装下面依赖

代码块

```
sudo apt install python3-dev python3-pip
sudo pip3 install websocket-client
```

- 运行脚本

代码块

```
python3 websocket_client.py
```

- websocket_client.py 实现

代码块

```
import json
import uuid
import threading
import time
import websocket
from datetime import datetime
```

```python

# Replace this ACCID value with your robot's actual serial number (SN)
ACCID = None

# Replace it with the real IP address of the robot.
# for a real machine, it is: 10.192.1.2
ROBOT_IP = "10.192.1.2"

# Atomic flag for graceful exit
should_exit = False

# WebSocket client instance
ws_client = None

# Generate dynamic GUID
def generate_guid():
    return str(uuid.uuid4())

# Send WebSocket request with title and data
def send_request(title, data=None):
    global ACCID
    if data is None:
        data = {}

    # Create message structure with necessary fields
    message = {
        "accid": ACCID,
        "title": title,
        "timestamp": int(time.time() * 1000),  # Current timestamp in milliseconds
        "guid": generate_guid(),
        "data": data
    }

    message_str = json.dumps(message)

    # Send the message through WebSocket if client is connected
    if ws_client:
        ws_client.send(message_str)

# Handle user commands
def handle_commands():
    global should_exit
    while not should_exit:
        command = input("Enter command ('movej', 'movep', 'light', 'stop') or 'exit' to quit:\n")

```

```python
        if command == "exit":
            should_exit = True  # Set exit flag to stop the loop
            break
        elif command == "movej":
            send_request("request_movej", { # request_movej
              "joint": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ,0],
              "time": 5
            })
        elif command == "movep":
            send_request("request_movep", { # request_movep
              "pos": [0.3, 0.2, -0.3, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0.3, -0.2, -0.3, 1, 0, 0, 0, 1, 0, 0, 0, 1],
              "time": 5
            })
        elif command == "light":
            send_request("request_light_effect", { # request_light_effect
              "effect": 1
            })
        elif command == "stop":
            send_request("request_emgy_stop", { # request_emgy_stop
            })

# WebSocket on_open callback
def on_open(ws):
    print("Connected!")
    # Start handling commands in a separate thread
    threading.Thread(target=handle_commands, daemon=True).start()

# WebSocket on_message callback
def on_message(ws, message):
    global ACCID
    root = json.loads(message)
    title = root.get("title", "")
    ACCID = root.get("accid", None)

    if title != "notify_robot_info":
        print(f"Received message: {message}")  # Print the received message

# WebSocket on_close callback
def on_close(ws, close_status_code, close_msg):
    print("Connection closed.")

# Close WebSocket connection
def close_connection(ws):
    ws.close()

def main():
```

```
    global ws_client

    # Create WebSocket client instance
    ws_client = websocket.WebSocketApp(
        f"ws://{ROBOT_IP}:5000",  # WebSocket server URI
        on_open=on_open,
        on_message=on_message,
        on_close=on_close
    )

    # Configure socket send and receive buffer sizes
    # Increase send buffer size to 2MB (default is typically much smaller)
    # This helps prevent data loss when sending large messages or high-frequency data
    ws_client.sock_opt = [("socket", "SO_SNDBUF", 2 * 1024 * 1024)]

    # Increase receive buffer size to 2MB
    # This allows handling larger incoming messages without truncation
    ws_client.sock_opt.append(("socket", "SO_RCVBUF", 2 * 1024 * 1024))

    # Run WebSocket client loop
    print("Press Ctrl+C to exit.")
    ws_client.run_forever()

if __name__ == "__main__":
    main()

```

4. 外接部件开发接口

4.1 逐际二指夹爪

4.1.1 夹爪控制指令

4.1.1.1 请求：request_set_limx_2fclaw_cmd

此协议控制夹爪的抓取动作。

代码块

```
{
  "accid": "DA_TRON2A_001",
  "title": "request_set_limx_2fclaw_cmd",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
```

```
  "data": {
      # 如您同时给出以下数据，则会控制左夹爪运动
      "left_opening": 50,  # 开口度，0-100，无量纲（0对应最小闭合，100对应张开到最大）
      "left_speed": 50,    # 夹爪速度，0~100 无量纲（数值越大速度越快）
      "left_force": 50,   #力，夹爪夹持力，0~100 无单位（数值越大力越大）

      # 如您同时给出以下数据，则会控制右夹爪运动
      "right_opening": 50,  # 开口度，0-100，无量纲（0对应最小闭合，100对应张开到最大）
      "right_speed": 50,    # 夹爪速度，0~100 无量纲（数值越大速度越快）
      "right_force": 50   #力，夹爪夹持力，0~100 无单位（数值越大力越大）
  }
}

```

4.1.1.2 响应：response_set_limx_2fclaw_cmd

代码块

```
{
  "accid": "DA_TRON2A_001",
  "title": "response_set_limx_2fclaw_cmd",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
    "result": "success"  # success: 成功, fail_motor: 电机错误
  }
}
```

4.1.1.3 消息推送：无

4.1.2 获取夹爪状态信息

4.1.2.1 请求：request_get_limx_2fclaw_state

代码块

```
{
  "accid": "DA_TRON2A_001",
  "title": "request_get_limx_2fclaw_state",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
  }
```

```
}
```

4.1.2.2 响应：response_get_limx_2fclaw_state

返回夹爪状态信息

代码块

```
{
  "accid": "DA_TRON2A_001",
  "title": "response_get_limx_2fclaw_state",
  "timestamp": 1672373633989,
  "guid": "746d937cd8094f6a98c9577aaf213d98",
  "data": {
      "timestamp": 1672373633989, # 表示数据时戳，单位为毫秒

      "left_opening": 50,  # 开口度，0-100，无量纲（0对应最小闭合，100对应张开到最大）
      "left_speed": 50,    # 夹爪速度，0~100 无量纲（数值越大速度越快）
      "left_force": 50,   #力，夹爪夹持力，0~100 无单位（数值越大力越大）

      "right_opening": 50,  # 开口度，0-100，无量纲（0对应最小闭合，100对应张开到最大）
      "right_speed": 50,    # 夹爪速度，0~100 无量纲（数值越大速度越快）
      "right_force": 50,   #力，夹爪夹持力，0~100 无单位（数值越大力越大）

      "result": "success"  # success: 成功, fail_motor: 电机错误
  }
}
```

4.1.2.3 消息推送：无

5. VR数采

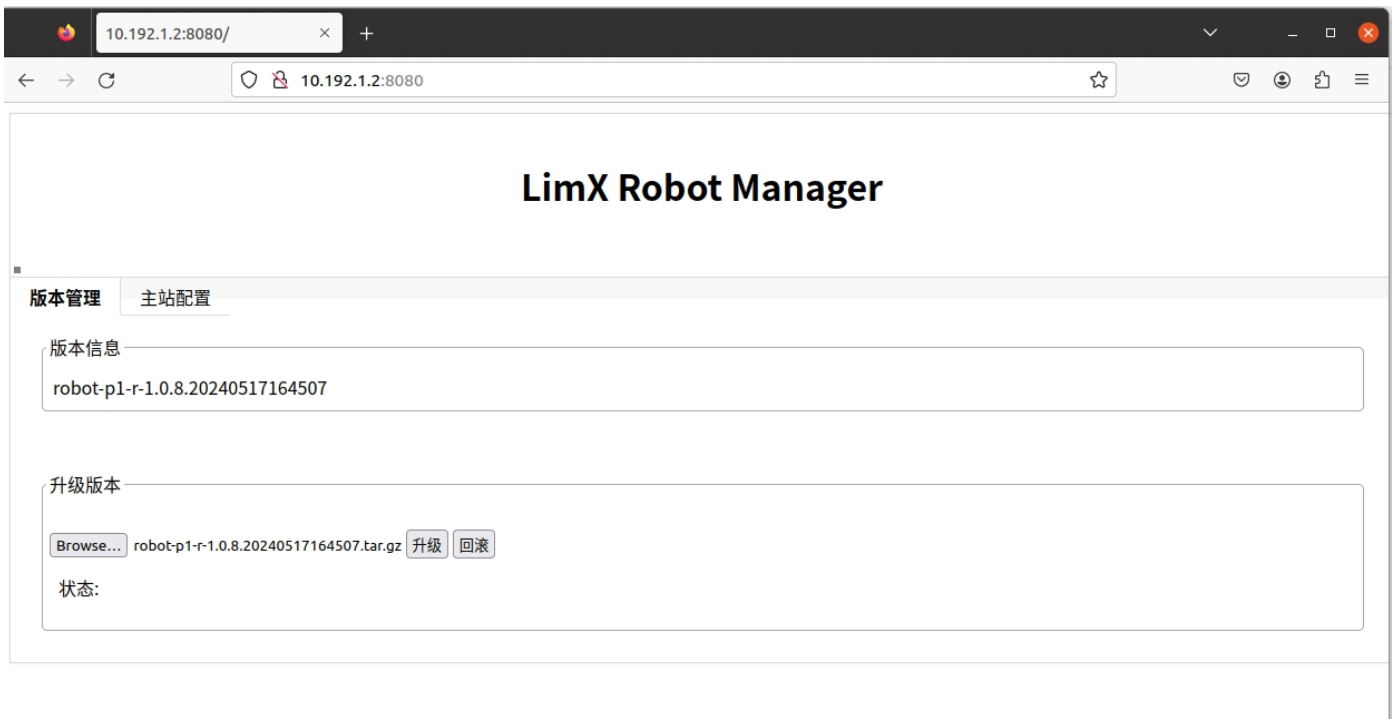
数据采集软件使用手册

6. 软件升级

6.1 机器人固件升级

我们通过浏览器进入机器人管理页面，选择本地提前下载好的机器人软件版本进行升级。具体步骤如下：

1. 请选择并连接您机器人的 Wi-Fi 热点，密码为：`12345678`
2. 使用Shell命令 `ping 10.192.1.2` 以确保连接正常
3. 访问管理页面：
   - 在浏览器地址栏输入：http://10.192.1.2:8080 进入机器人管理页面。
4. 选择和升级软件：
   - 依次选择“版本管理 -> 选择文件 -> 升级”。
   - 升级完成后，机器人主控电脑将自动重启。



Tron2（v4）固件包

软件类别	版本号	发布日期	软件名称	软件包	更新说明
主控软件版本	r-1.2.25	20260113	robot-tron2-r-1.2.25.20260107211002.tar.gz	robot-tron2… 306.44MB	
主站板版本	1.0.25	20260113	ecm_V1.0.25_20251230.tar.gz	ecm_V1.0.25… 80.87MB	
驱动器版本	1.2.23	20260113	EcMotor_HU_V1.2.23_20251202_174735.bin	EcMotor_HU… 106.09KB	

分电板版本	**1.0.4**	20260113	**PMSGen2_T2_V1.0.4_20251204.bin**	PMSGen2_T… 144.91KB	
遥控器版本	**1.0.2**	20251016	**RCGen2_T2_V1.0.2_20250919**	RCGen2_T2_… 236.04KB	
studio版本	**0.1.39**	20260113	**V0.1.39**	limx-studi… 129.35MB	**说明：studio0.1.39必须与主控软件版本1.2.25搭配使用！** Studio桌面端功能项： 1. Studio新增TRON2设备管理功能，支持轮足形态、脚掌形态、双臂形态。 2. Studio TRON2新增真机管理模块，显示基本信息、设备信息、电量信息。 3. Studio TRON2新增版本管理模块，显示版本信息、升级管理，支持软件包上传升级。 4. Studio TRON2新增机器人设置模块，显示摄像头角度设置、桌面高度识别、摄像头角度控制及弧度控制、头部跟随VR功能。 5. 需求变更：功能调整，隐藏桌面高度识别功能。 6. 添加移动摄像头，弧度回显。 7. 调整机器人设置布局，增加兼容性。

6.2 VR遥操软件升级

1. 使用type-C线将quest3与电脑连接，使用手柄点击usb弹窗，允许电脑访问quest3内部数据
2. 下载遥操软件到您的电脑

LimXTeleopTronV4_3.apk
64.51MB

3. 在您的电脑中安装adb工具

代码块

```
sudo apt install android-tools-adb
```

安装结束后，可以输入命令 `adb devices` 查看是否已连接到quest3

4. 安装遥操软件到quest3设备

代码块

```
adb install -r LimXTeleopTronV4_3.apk
```

7. 常见问题

序号	问题	常见原因	解决方式
1	访问 `http://10.192.1.2:8080` 没反应	电脑不在10.192.1.2同网段	1. 先ping10.192.1.2，通了清除下浏览器数据，或者换浏览器重新访问 2. 没ping通就修改本机地址，关闭dhcp，DNS为223.5.5.5 或者 8.8.8.8
2	如何配置相机sn号		相机SN配置方法
3			
4			
5			

深圳逐际动力科技有限公司

地址：深圳市南山区沙河西路3157号南山智谷产业园E栋15层

网址：https://limxdynamics.com/